



Enable Python and PowerShell Without Losing Control

| Josh More, IANS Faculty

The Takeaway

Many modern tools presume that Python and PowerShell are available in an environment and anyone wanting to automate common admin tasks or perform data analysis can request access to such tools. Fortunately, while the era of “just block it” is over, it is possible to tier use cases and provide progressively stronger training and gating controls to match the type of work that people want to do. This report explains how to allow limited access to Python and PowerShell and make sure that approvals are fast and complete, enabling you to walk the line between blocking vital tooling and allowing unfettered use.

The Challenge

A security team in government is facing pressure to open their restricted environment for scripting tools. [PowerShell](#) is wanted to manage permissions within [Microsoft SharePoint](#). [Python](#) users want to expand into analytics and forecasting. Specifically, the team asks:

- What risks should we consider when enabling end users to incorporate Python and PowerShell into daily workflows, particularly for analytics, forecasting and administrative tasks?
- What governance, access controls and monitoring practices are best practice for safely enabling scripting tools across end-user populations in a state government environment?
- How should we determine appropriate levels of access and responsibility for end users when expanding scripting capabilities from restricted to permitted use?

Moving Beyond the Binary

When managing access to scripting tools in the past, organizations focused on a simple binary: restricted versus permitted. However, when people need additional capabilities to do their job, but you are still concerned about abuse of those privileges, you need a more nuanced approach. This is doubly true in a world where AI is rapidly enabling non-programmers to develop scripts in these languages, and triply true where attackers use “living off the land” techniques to fully abuse the privileges that people have.

Security must focus on enablement without a fully permissive model. Options include techniques such as approved interpreters, approved library sources, an approved way to perform a SharePoint permission change, and—as always—a quick way to approve exceptions when they occur.

Understand the Risks

The PowerShell and Python risks are different, so there is no single solution here.

The issue with PowerShell is living-off-the-land attacks. Because PowerShell executables are signed, present everywhere and trusted, attackers can use PowerShell directly instead of dropping a binary and getting caught

by application control or endpoint detection and response (EDR). Worse, when used in an environment with SharePoint, permissions can have an unexpected blast radius. A single script can change sharing permissions on a site collection or, ransomware style, remove access from hundreds of items in seconds. Then there are, of course, the standard failure modes: credential and token exposure and blind spots where logging and [Constrained Language Mode](#) (CLM) are not enabled.

For Python, the concern is the supply chain. From typosquatted packages, dependency-confusion, to the newer AI coding assistant issues recommending non-existent packages that attackers can create—the risks are mostly within the global Python development community. Beyond the supply chain, the other risk is legacy code that no one understands—often left behind when people leave the organization and rarely reviewed, but that AI can find and use with little warning.

Controlling PowerShell

[App Control for Business](#) (formerly [Windows Defender Application Control](#), or WDAC) is now Microsoft's preferred application-control path. [AppLocker](#) has moved to maintenance mode and is [now receiving security fixes only](#). So today, your best bet is to use App Control to enforce CLM, a move that was reinforced when a security researcher discovered that AppLocker-enforced CLM silently broke for several months in early 2025, allowing scripts to run in full language mode when they should have been contained. App Control works at a deeper level and environments using it did not experience that gap.

Before you deploy, set up a test group and enforce CLM through App Control, using PowerShell 7.4's audit mode to find what breaks. It is critical to do this testing exhaustively, as a single unsigned piece could cause the whole script to fail—likely breaking many existing workflows.

As you do this, be sure to enable script block logging, module logging and transcription—and make sure that the PowerShell operational log logs enough to help you troubleshoot. (500 MB is a good target, if you have the space; pushing to a SIEM is also wise.) Script block logging is verbose, and a low log size limit will cause the log to roll over and discard the evidence you turned it on to capture.

Separately, remove the PowerShell 2.0 engine. This downgrade path allows for a bypass of the [Antimalware Scan Interface](#) (AMSI), and odds are that nothing in your environment needs it.

SharePoint Specifics

For the SharePoint permission use case, don't give end users interactive PowerShell. Instead, create a scoped automation identity. Specifically, make an [Entra](#) application registration granted Sites.Selected and consented per site, not Sites.FullControl.All. You need to keep it scoped small because a tenant-wide grant allows a single script to access and share documents across the entire tenant, so if there is a bug or security failure, everything is at risk. However, per-site consent limits any mistake to the sites the script is intended to touch.

This approach is enhanced with the just in time (JIT)/just enough administration (JEA) principle, allowing a human to run curated commands instead of an unattended and uncontrolled process, exposing only the specific permission cmdlets needed. Additionally, properly set up logging will track every action in the transcript. JIT/JEA is underused because it can be tricky to set up, but this is the exact use case that it is designed for, and it's worth the effort.

If you face budget or time constraints, start with the logging and the PowerShell 2.0 removal. It won't cover everything but can make the difference between an incident you can investigate and one you can't.



Controlling Python

When working in Python, focus on the supply chain before you look at the endpoint. Start by routing pip through an internal mirror of some sort such as [JFrog](#), [Artifactory](#), [Sonatype Nexus](#), [Azure Artifacts](#) and [AWS CodeArtifact](#). You can do it yourself as well, through self-hosted [devpi](#). Then block access to pypi.org (and any other sources you may connect to the mirror) from all other locations. Use `--index-url` to point only at the mirror and do not use (and monitor for others using) `--extra-index-url`, as it could bypass your controls and access public resources.

Then, pin your dependencies by hash (`pip-compile --generate-hashes`) and generate a software bill of materials so you know what's included. Then you can connect it to your preferred software composition analysis (SCA) scanning tool. If you cannot stand up the full pipeline in one day, mirror-plus-hash pinning alone will buy you a lot of risk reduction (though approved packages that become compromised later will be missed). You can add scanning later.

Before restricting anything, run a discovery. Use your existing EDR or application-control telemetry to find who is running the interpreter and from where. Even if you think you have it blocked, you may be surprised by what's out there. Do not rely on a spreadsheet inventory for this. It will be out of date and almost certainly not cover the data you want.

For the analytics and forecasting part, the lowest-risk option is to keep a general interpreter off the endpoint entirely. [Python in Excel](#) runs [pandas](#), [scikit-learn](#) and [Matplotlib](#) in a per-workbook. A hypervisor-isolated [Azure](#) container with no access to the local machine can use [Anaconda](#) code and get its packages from a curated Anaconda source. The code physically cannot reach the endpoint, and a workbook opened from the internet will not run its Python at all. If the forecasting data already lives in [Snowflake](#), native Python user-defined functions with a Packages Policy give you an allowlist and version pinning without buying anything new. You can also set up a dedicated analytics machine with tight access controls and limited access to data, to minimize blast radius should anything go wrong.

Whatever you build, keep the approved path as fast as you can. Analysts will route around a slow pipeline, bypassing your controls. If approval takes more than a day, you will likely find unapproved packages in production use. Speed of the clean path is not a nicety here. It is the control.

Tier the Access

The rest of the question, how much access and how it is gated, is better answered by population and path—not just by job title.

Tier 1: Sandboxed analytics, focused on analysts and forecasters doing math on data

For this type of use, use Python in Excel or Snowflake Python functions. Use the allowlist native in each platform and any supported sensitivity labels. Do not allow access to a local interpreter. There is no need for a test or certification for this use.

Tier 2: Approved scripting, focused on power users and admins automating defined tasks

For this approach, use a local interpreter, but restrict packages to only those coming from the internal mirror. Logging should be forced on, and no one using this should have standing admin credentials. Require a short competency check to get this level of access, with a named owner for each script and a process to change that owner should the named owner change roles or leave the organization.

Tier 3: Automation authors, focused on code that runs unattended or changes permissions at scale

This use case requires formalized source control, peer review, code signing and SCA in the pipeline before anything runs with production data. This is where you require training and certification, as it is aligned to the real-world risk and lowers the barrier for less risky activities. As in Tier 2, record a named human owner for every script and automation. Orphaned automation whose author is no longer with the organization is a sure way to get an audit finding.

Monitor the Program

Monitor what you can act on. For PowerShell, watch for script block events (Event ID 4104) and CLM state, code integrity policy events and SharePoint permission-change events. For Python, monitor for unmanaged pip calls and egress attempts to public package sources as well as any interpreter executing from a user-writable path. Route these to the SIEM, and alert specifically on event-log clears and on execution from user-writable directories, since both are strong early signals.

There are public baselines you can use. The Defense Information Systems Agency (DISA) PowerShell [Security Technical Implementation Guide](#) (STIG) and the [Center for Internet Security](#) (CIS) both provide benchmarks for [Windows](#) and [Microsoft 365](#). If you expect to be audited on these controls, adopting an existing benchmark works better than trying to develop and defend one that is home grown.

There is also a useful line in the sand you can use. If a user writes code that other people depend on, and that cannot be worked around manually if it fails, that counts as true software development and would fall under your SDLC, not any of the tiers mentioned above.

Manage the Rollout

Plan to enable Python and PowerShell in waves. Counter to a lot of security advice, do not take a risk-based approach. Instead, focus on ease of setup and how well the process can be containerized. Look at shared and administrative systems, and the SharePoint automation identity, setting up repeatable, restricted workflows. Let these stabilize, and then work up the tiers, to Tier 2 and then Tier 3.

Do not set this up across the entire environment on day one. That's a good way to fail and spend your next several weeks dealing with false blocks and burn all the good will you have going into the project. Instead, start small and move slow, verifying that every step you take works as expected both from a functionality and a security control perspective before going on to the next step.

Follow Three Core Principles

As you enable access to Python and PowerShell, keep in mind the following three principles:

- **Enable the path:** The team's job is to create a fast, approved path. Implementing a tight restriction only drives people to bypass your controls.
- **Control both the interpreter and the supply chain:** Restricting access to the interpreters, the environments in which they can execute and the components they can leverage, covers most of the risk.
- **Align rigor to potential blast radius:** As you go up the tiers, use additional gates to ensure that appropriate controls are in place for each use case.

It will take time to build controlled access to Python and PowerShell. Stand up internal mirrors, turn on logging, scope the SharePoint automations and route analyst work to analysis platforms. By doing this, you can address most of the risks before you turn people loose on them. The rest is about managing your people and continued refinement of your controls.

Further Reading

[Governing Shadow Python and AI Development in Financial Services](#), June 2, 2026

[Best Practices for Securing PowerShell and Command Prompt Usage](#), Nov. 20, 2025



Josh More
IANS Faculty

Josh is the owner and president of Eyra Security, an information security and business improvement consulting firm that specializes in helping startups and organizations in transition take advantage of lean and agile methods, open source technology and varied frameworks used for security, risk management and compliance. He also serves on the GIAC Advisory Board. [Learn more.](#)

Any views or opinions presented in this document are solely those of the Faculty and do not necessarily represent the views and opinions of IANS. Although reasonable efforts will be made to ensure the completeness and accuracy of the information contained in our written reports, no liability can be accepted by IANS or our Faculty members for the results of any actions taken by the client in connection with such information, opinions, or advice.